

Core Language Working Group "tentatively ready" Issues for the November, 2020 virtual meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4868](#).

2312. Structured bindings and mutable

Section: 9.6 [dcl.struct.bind] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-08-11

An example like the following is currently ill-formed:

```
struct A { mutable int n; };  
void f() {  
    const auto [a] = A();  
    a = 0;  
}
```

According to 9.6 [dcl.struct.bind] paragraph 4, the type of `a` is `const int`, since the implicitly-declared variable is `const`. This seems obviously wrong: the member `n` is mutable, so the member access expression `e.n` has type `int`, which should also be the type of `a`. (`mutable` should presumably be taken into account when forming the referenced type too, so that `decltype(a)` is `int` as would presumably be expected, rather than `const int`.)

Proposed resolution, March, 2018: [SUPERSEDED]

Change 9.6 [dcl.struct.bind] paragraph 4 as follows:

...Designating the non-static data members of `E` as `m0, m1, m2, ...` (in declaration order), each `vi` is the name of an lvalue that refers to the member `mi` of `e` and whose type is `cv Ti`, where `Ti` is the declared type of that member `e.mi`; the referenced type is `cv Ti` the type of `e.mi`. The lvalue is a bit-field if...

Notes from the June, 2018 meeting:

It was observed that this resolution does not handle members with reference type correctly. The main problem seems to be the statement in 7.6.1.5 [expr.ref] paragraph 4, which directly handles members with reference type rather than allowing the type of the member to be the result type and relying on the general rule that turns reference-typed expressions into lvalues.

Proposed resolution (April, 2020):

Change 9.6 [dcl.struct.bind] paragraph 5 as follows:

...Designating the non-static data members of `E` as `m0, m1, m2, ...` (in declaration order), each `vi` is the name of an lvalue that refers to the member `mi` of `e` and whose type is `cv Ti`, where `Ti` is the declared type of that

member that of *e.m_i*; (7.6.1.5 [expr.ref]); the referenced type is *cv-T_i*; the declared type of *m_i* if that type is a reference type, or the type of *e.m_i*; otherwise. The lvalue is a bit-field if that member is a bit-field.

[Example 2:

```
struct S { mutable int x1 : 2; volatile double y1; };
S f();
const auto [ x, y ] = f();
```

The type of the *id-expression* *x* is “const int”, the type of the *id-expression* *y* is “const volatile double”. —end example]

2369. Ordering between constraints and substitution

Section: 13.10.3 [temp.deduct] **Status:** tentatively ready **Submitter:** Agustin Bergé **Date:** 2017-10-09

The specification of template argument deduction in 13.10.3 [temp.deduct] paragraph 5 specifies the order of processing as:

1. substitute explicitly-specified template arguments throughout the template parameter list and type;
2. deduce template arguments from the resulting function signature;
3. check that non-dependent parameters can be initialized from their arguments;
4. substitute deduced template arguments into the template parameter list and particularly into any needed default arguments to form a complete template argument list;;
5. substitute resulting template arguments throughout the type;
6. check that the associated constraints are satisfied;
7. check that remaining parameters can be initialized from their arguments.

This ordering yields unexpected differences between concept and SFINAE implementations. For example:

```
template <typename T>
struct static_assert_integral {
    static_assert(std::is_integral_v<T>);
    using type = T;
};

struct fun {
    template <typename T,
        typename Requires = std::enable_if_t<std::is_integral_v<T>>>
        typename static_assert_integral<T>::type
    operator()(T) {}
};
```

Here the substitution ordering guarantees are leveraged to prevent `static_assert_integral<T>` from being instantiated when the constraints are not satisfied. As a result, the following assertion holds:

```
static_assert(!std::is_invocable_v<fun, float>);
```

A version of this code written using constraints unexpectedly behaves differently:

```
struct fun {
    template <typename T>
    requires std::is_integral_v<T>
```

```

    typename static_assert_integral<T>::type
    operator()(T) {}
};

```

or

```

struct fun {
    template <typename T>
    typename static_assert_integral<T>::type
    operator()(T) requires std::is_integral_v<T> {}
};

```

```

static_assert(!std::is_invocable_v<fun, float>); // error: static assertion failed: std::is_integral_v<T>

```

Perhaps steps 5 and 6 should be interchanged.

Proposed resolution (August, 2020):

1. Delete paragraph 10 of 13.10.3.2 [temp.deduct.call]:

~~If deduction succeeds for all parameters that contain *template parameters* that participate in template argument deduction, and all template arguments are explicitly specified, deduced, or obtained from default template arguments, remaining parameters are then compared with the corresponding arguments. For each remaining parameter P with a type that was non-dependent before substitution of any explicitly-specified template arguments, if the corresponding argument A cannot be implicitly converted to P, deduction fails. [Note 2: Parameters with dependent types in which no *template parameters* participate in template argument deduction, and parameters that became non-dependent due to substitution of explicitly-specified template arguments, will be checked during overload resolution. —end note]~~

~~[Example 9:~~

```

template <class T> struct Z {
    typedef typename T::x xx;
};
template <class T> typename Z<T>::xx f(void *, T); // #1
template <class T> void f(int, T); // #2
struct A { } a;
int main() {
    f(1, a); // OK, deduction fails for #1 because there is no conversion from int to void*
}

```

~~—end example]~~

2. Change 13.10.3.1 [temp.deduct.general] paragraph 5 as follows:

...When all template arguments have been deduced or obtained from default template arguments, all uses of template parameters in the template parameter list of the template ~~and the function type~~ are replaced with the corresponding deduced or default argument values. If the substitution results in an invalid type, as described above, type deduction fails. If the function template has associated constraints (13.5.3 [temp.constr.decl]), those constraints are checked for satisfaction (13.5.2 [temp.constr.constr]). If the constraints are not satisfied, type deduction fails. **In the context of a function call, if type deduction has not yet failed, then for those function parameters for which the function call has arguments, each function parameter with a type that was non-dependent before substitution of any explicitly-specified template arguments is checked against its corresponding argument; if the corresponding argument cannot be implicitly converted to the parameter type, type deduction fails. [Note: Overload resolution will check the other parameters, including parameters with dependent types in which no template parameters participate in template argument deduction and parameters that became non-dependent due to substitution of explicitly-specified template arguments. —end note] If type deduction has not yet failed, then all uses of template parameters in the function type are replaced with the**

corresponding deduced or default argument values. If the substitution results in an invalid type, as described above, type deduction fails. [Example:

```
template <class T> struct Z {
    typedef typename T::x xx;
};
template <class T> concept C = requires { typename T::A; };
template <C T> typename Z<T>::xx f(void *, T); // #1
template <class T> void f(int, T); // #2
struct A {} a;
struct ZZ {
    template <class T, class = typename Z<T>::xx> operator T *();
    operator int();
};
int main() {
    ZZ zz;
    f(1, a); // OK, deduction fails for #1 because there is no conversion from int to void*
    f(zz, 42); // OK, deduction fails for #1 because C<int> is not satisfied
}

—end example]
```

2452. Flowing off the end of a coroutine

Section: 8.7.5 [stmt.return.coroutine] **Status:** tentatively ready **Submitter:** Lewis Baker **Date:** 2020-02-14

There are two references to “flowing off the end of a coroutine”, specifically in 8.7.5 [stmt.return.coroutine] paragraph 3:

If `p.return_void()` is a valid expression, flowing off the end of a coroutine is equivalent to a `co_return` with no operand; otherwise flowing off the end of a coroutine results in undefined behavior.

and 9.5.4 [dcl.fct.def.coroutine] paragraph 11:

The coroutine state is destroyed when control flows off the end of the coroutine or...

These mean different things and should be clarified.

Proposed resolution (July, 2020):

Change 8.7.5 [stmt.return.coroutine] paragraph 3 as follows:

If `p.return_void()` is a valid expression, flowing off the end of a coroutine's **function-body** is equivalent to a `co_return` with no operand; otherwise flowing off the end of a coroutine's **function-body** results in undefined behavior.

2457. Unexpanded parameter packs don't make a function type dependent

Section: 13.8.3.2 [temp.dep.type] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2020-07-28

Consider the following example:

```
template<typename ...T> auto f() {
    using F = int(*) (int (...p)[sizeof(T)]);
    // ...
}
```

}

F is not covered in the list of cases in 13.8.3.2 [temp.dep.type] paragraph 9, because the types from which the function type is constructed are not dependent types. (The parameter pack `p` is of type `int[sizeof(size_t)]`.) Similar situations arise with non-injective alias templates.

Proposed resolution (August, 2020):

Change 13.8.3.2 [temp.dep.type] paragraph 9 as follows:

A type is dependent if it is

- ...
- an array type whose element type is dependent or whose bound (if any) is value-dependent,
- **a function type whose parameters include one or more function parameter packs,**
- a function type whose exception specification is value-dependent,
- ...

(We do have the relevant wording for pack expansions in *simple-template-ids* in bullet 9.8, so that similar case is already handled.)

2460. C language linkage and constrained non-template friends

Section: 9.11 [dcl.link] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2020-03-23

According to 13.7.5 [temp.friend] paragraph 9,

A non-template friend declaration with a *requires-clause* shall be a definition. A friend function template with a constraint that depends on a template parameter from an enclosing template shall be a definition. Such a constrained friend function or function template declaration does not declare the same function or function template as a declaration in any other scope.

However, this specification conflicts with the treatment of functions with C language linkage in 9.11 [dcl.link] paragraph 7:

At most one function with a particular name can have C language linkage. Two declarations for a function with C language linkage with the same function name (ignoring the namespace names that qualify it) that appear in different namespace scopes refer to the same function.

For example:

```
template <typename T> struct A { struct B; };

extern "C" {
    template <typename T>
    struct A<T>::B {
        friend void f(B *) requires true {} // C language linkage applies
    };
}

namespace Q {
    extern "C" void f(); // ill-formed redeclaration?
}
```

Proposed resolution (April, 2020):

Change 9.11 [dcl.link] paragraph 5 as follows:

...A C language linkage is ignored in determining the language linkage of the names of class members, **the names of friend functions with a trailing *requires-clause***, and the function type of class member functions...

2461. Diagnosing non-bool type constraints

Section: 13.8 [temp.res] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2020-04-20

Given the following example,

```
template <typename T> struct A {};  
template <typename T> void f() requires (sizeof(A<T>)) {}
```

the current wording does not appear to allow diagnosis of the program as ill-formed. In particular, 13.8 [temp.res] bullet 8.2 says,

The program is ill-formed, no diagnostic required, if:

- ...
- no substitution of template arguments into a *type-constraint* or *requires-clause* would result in a valid expression, or
- ...

However, substitution into the *requires-clause* in this case would result in a valid expression, but not one that is an atomic constraint that can be checked for satisfaction.

Proposed resolution (April, 2020):

Change bullet 8.2 of 13.8 [temp.res] as follows:

The program is ill-formed, no diagnostic required, if:

- ...
- ~~no substitution of template arguments into a *type-constraint* or *requires-clause* would result in a valid expression~~ **any *constraint-expression* in the program, introduced or otherwise, has (in its normal form) an atomic constraint *A* where no satisfaction check of *A* could be well-formed and no satisfaction check of *A* is performed**, or
- ...